

Project Design Document

School: Computational Sciences and Artificial Intelligence

Course: CSAI 101: Fundamentals of Programming and Computer Science

Team Members:

- **Youssef:** Newton's Method Module
- **Hassan:** Vector Library Module
- **Mahmoud:** Calculus Utilities Module
- **All Members:** Graphical User Interface (GUI) Integration

1. Function Directory Table

The following table inventories all operational functions developed across the program modules, assigning tracking, data constraints, and implementation responsibilities:

Function Name	Input Parameters	Output (Return)	Task Description	Team Member
<code>_prepare_expression</code> (<i>newton_method.py</i>)	<code>equation: str</code> <code>variable: str</code>	<code>Tuple[Expression, Symbol]</code>	Parses mathematical equation strings and maps custom algebraic symbols (e.g., matching 'e' to SymPy's E). Handles equality symbols (=) by equating the equation to zero.	Youssef
<code>_as_real_number</code> (<i>newton_method.py</i>)	<code>value: Any</code>	<code>float</code>	Evaluates evaluated SymPy expressions into float data types. Raises a <code>ValueError</code> if an operation evaluates to an	Youssef

			unexpected complex number.	
<code>newton_method</code> <i>(newton_method.py)</i>	<code>equation: str</code> <code>initial_guess : Any</code> <code>variable: str</code> <code>tolerance: float</code> <code>max_iteratio ns: int</code>	<code>Tuple[float, int]</code>	Approximates a single function root utilizing single-variable Newton-Raphson iterations. Includes safeguards against zero- division errors.	Youssef

<code>find_roots</code> <i>(newton_method.py)</i>	<code>equation: str</code> <code>guesses: Iterable</code> <code>variable: str</code> <code>tolerance: float</code> <code>max_iterations: int</code>	<code>List[Tuple[float, Any, int]]</code>	Batch-processes multiple root evaluations using a collection of starting boundary guesses, returning compiled unique convergent roots.	Youssef
<code>_prepare_expression</code> <i>(library.py)</i>	<code>function: Any</code> <code>variable: str</code>	<code>Tuple[Expression, Symbol]</code>	Prepares single string-based mathematical structures to parse cleanly inside algebraic calculus environments.	Mahmoud

<code>valid_function</code> <i>(library.py)</i>	<code>function:</code> <code>Any</code>	<code>Expression</code>	Sanitizes structural inputs to verify compliance, explicitly rejecting expressions configured with equation constraints (<code>=</code>).	Mahmo ud
<code>definite_integral</code> <i>(library.py)</i>	<code>function:</code> <code>Any</code> <code>x1: Any</code> <code>x2: Any</code> <code>variable: str</code>	<code>float</code>	Computes formal single-variable definite integrations within bounding values (<code>x_1</code>) and (<code>x_2</code>) via standard anti-derivative parsing.	Mahmo ud

<code>area_between_2functions</code> <i>(library.py)</i>	<code>function1:</code> <code>Any</code> <code>function2:</code> <code>Any</code> <code>x1:</code> <code>Any</code> <code>x2:</code> <code>Any</code> <code>variable:</code> <code>str</code>	<code>float</code>	Computes total geometric area trapped between two functions. Automatically checks and splits boundaries at intersection turning points using absolute integration.	Mahmoud
<code>Tangent_Line</code> <i>(library.py)</i>	<code>function:</code> <code>Any</code> <code>point:</code> <code>Any</code> <code>variable:</code> <code>str</code>	<code>str</code>	Determines the precise tangent line coordinate equation ($y = mx + c$) intercepting a mathematical curve at a requested coordinate point.	Mahmoud

<u>Normal_line</u> <i>(library.py)</i>	<u>function:</u> <u>Any</u> <u>point: Any</u> <u>variable: str</u>	<u>str</u>	Extracts the exact normal perpendicular slope equation line mapping out from a functional coordinate intersection.	Mahmo ud
<u>_to_numbers</u> <i>(vector_library.py)</i>	<u>vector: Any</u>	<u>List[float]</u>	Transforms primitive matrix array types into raw native system floats for clean linear algebra manipulations.	Hassan
<u>_pad_vectors</u> <i>(vector_library.py)</i>	<u>vector1: Any</u> <u>vector2: Any</u>	<u>Tuple[List[float], List[float]]</u>	Validates vector layouts to verify space constraints (2D/3D). Pads 2D structures out with 0.0 placeholder dimensions to safely evaluate cross products.	Hassan

vector_coordinates <i>(vector_library.py)</i>	vector: Any	str	Formats multi-dimensional numeric data arrays back into structural Cartesian notations using standard engineering units.	Hassan
magnitude <i>(vector_library.py)</i>	vector: Any	float	Evaluates absolute geometric length or norm dimensions.	Hassan
angle_with_axis <i>(vector_library.py)</i>	vector: Any	float Tuple[float, ...]	Measures vector directional paths pointing from native primary reference coordinate lines in both planar 2D and spatial 3D spaces.	Hassan
dot <i>(vector_library.py)</i>	vector1: Any vector2: Any	float	Executes linear dot-product tracking by accumulating combined spatial axis projections across	Hassan

			independent array inputs	
<code>angle_between_vectors</code> <i>(vector_library.py)</i>	<code>vector1: Any</code> <code>vector2: Any</code>	<code>float</code>	Isolates mutual spatial angular distances bridging distinct vectors via inverse cosine operations over scaled inner products.	Hassan
<code>cross_product</code> <i>(vector_library.py)</i>	<code>vector1: Any</code> <code>vector2: Any</code>	<code>float List[float]</code>	Generates perpendicular vector coordinates resulting from vector cross products. Evaluates planar determinants as standard scalar weights.	Hassan

<code>magnitude_of_cross</code> <i>(vector_library.py)</i>	<code>vector1: Any</code> <code>vector2: Any</code>	<code>float</code>	Captures raw magnitude scores generated out from computed cross products, indicating parallel spatial area tracking parameters.	Hassan
<code>format_number</code> <i>(gui.py)</i>	<code>value: Any</code>	<code>str</code>	Utility that rounds floating-point results to a high-precision layout or integer variant if trailing decimals fall inside small error thresholds.	Shared
<code>parse_vector</code> <i>(gui.py)</i>	<code>text: str</code>	<code>List[float]</code>	Formats string entries from raw GUI text input frames, stripping trailing spaces and separating inputs by commas or spaces into standard arrays.	Shared

	<code>columnspan</code> : int			
<code>_build_vector_tab</code> (gui.py)	<code>tab: Any</code>	<i>None</i>	Constructs layout panels, drop-down configuration options, control paths, and display interfaces assigned explicitly to the Vector toolkit panel.	Shared
<code>_build_calculus_tab</code> (gui.py)	<code>tab: Any</code>	<i>None</i>	Initializes standard workspace layouts tracking equation functions, coordinate points, execution variables, integration boundaries, and operations.	Shared

<code>_run_vector_calculation</code> (gui.py)	None	None	Event loop listener that extracts vector strings, invokes target operations inside Hassan's <code>vector_library.py</code> , and updates the UI text view.	Shared
<code>_run_calculus_calculation</code> (gui.py)	None	None	Coordinates calculations parsing input arguments before routing tasks to Mahmoud's <code>library.py</code> for integration or curve updates.	Shared
<code>_run_newton_calculation</code> (gui.py)	None	None	Event framework pulling configured root-seeking parameters to evaluate iterative approximations using Youssef's <code>newton_method.py</code> .	Shared